HAI-DEF-2026-009

[250]HANZO INDUSTRIES

Quantum Key Distribution and Quantum Sensing Integration for Post-Quantum Blockchain Consensus and Threshold Cryptography

[180]TECHNICAL WHITE PAPER

Version 1.0 — February 27, 2026 | February 2026

Hanzo Team

Hanzo Industries Lux Industries Zoo Labs Foundation
research@hanzo.ai

CONFIDENTIAL — Not for public distribution.
Handle in accordance with applicable regulations.

Abstract

We present an architecture for integrating quantum key distribution (QKD) hardware and quantum sensing devices into a production post-quantum blockchain stack. The system extends five existing production components—QuantumVM (Q-Chain), the KEM cryptographic interface, RNS mesh transport, Sovereign KMS, and the Hanzo Zero Trust fabric—with QKD-derived key material and quantum sensor attestations. A novel *triple-hybrid* key derivation combines classical Diffie-Hellman (X25519), post-quantum lattice encapsulation (ML-KEM-768, FIPS 203), and physics-layer QKD shared secrets via HKDF, ensuring link security if *any one* of three independent foundations holds: elliptic curve hardness, Module-LWE hardness, or the laws of quantum mechanics. Quantum sensors—atomic clocks, magnetometers, and gravimeters—feed cryptographically attested measurements into the Q-Chain via a Sensor Oracle, stamped with dual BLS/Ringtail threshold signatures for immutable provenance. We specify ETSI GS QKD 014 and ITU-T Y.3800 compliant interfaces, analyze integration with threshold MPC signing ceremonies, and project a 3,000–5,000 line implementation leveraging existing production interfaces. No competing blockchain system offers production post-quantum consensus, let alone QKD or quantum sensing integration.

Contents

1 Introduction **4**
1.1 Existing Production Foundation 4
1.2 What This Paper Adds 4

2 Quantum Key Distribution Integration **4**
2.1 QKD Protocol Support 4
2.2 ETSI GS QKD 014 Client 5
2.3 QKD-KEM Bridge 5

3 Triple-Hybrid Key Derivation **6**
3.1 Security Model 6
3.2 Protocol 6
3.3 Wire Format 7

4 Quantum Sensing and Attestation **7**
4.1 Sensor Types 7
4.2 Sensor Oracle Architecture 7
4.3 Atomic Clocks for Consensus Timing 7
4.4 Magnetometers for Physical Security 8
4.5 Gravimeters for Subsurface Intelligence 8

5 QRNG Integration **8**
5.1 Drop-In Entropy Source 8

6 Integration with Threshold MPC **9**
6.1 QKD-Authenticated DKG Ceremony 9
6.2 QKD-Secured Signing Sessions 9
6.3 Key Rotation with QKD Refresh 9

7 Integration with Consensus **10**
7.1 QKD-Authenticated Validator Links 10
7.2 Sensor-Attested Block Production 10

8 KMS Integration **10**
8.1 QKD Key Import . . . 10
8.2 Key Hierarchy with QKD Root . . . 11

9 Zero-Trust Fabric Integration **11**
9.1 QKD Transport Underlay . . . 11

10 Standards Compliance **11**

11 Naval Deployment Scenarios **11**
11.1 Shore-to-Ship QKD via Submarine Fiber . . . 11
11.2 Satellite QKD for Global Coverage . . . 12
11.3 Submarine Quantum Sensing Array . . . 12
11.4 GPS-Denied Navigation . . . 12

12 Implementation Roadmap **12**

13 Competitive Analysis **13**

14 Conclusion **13**

1 Introduction

Post-quantum cryptography protects against *algorithmic* quantum attacks—Shor's algorithm breaking RSA and elliptic curves, Grover's algorithm weakening symmetric ciphers. However, algorithmic security rests on computational hardness *assumptions* (Module-LWE, Module-SIS) that, while believed strong, lack the unconditional guarantee of physics-based security.

Quantum key distribution provides information-theoretic security grounded in the laws of quantum mechanics: an eavesdropper disturbing a quantum channel is detectable with probability approaching 1. Quantum sensing exploits quantum coherence and entanglement for measurement precision beyond classical limits.

This paper presents an architecture that layers physics-based security (QKD) atop algorithmic security (PQ crypto) atop classical security (ECDH), creating a triple-hybrid defense-in-depth unique in the blockchain and military networking landscape.

1.1 Existing Production Foundation

The architecture builds on five deployed systems:

1. **QuantumVM (Q-Chain)**: Production VM with ML-DSA-44/65/87 signing, SLH-DSA stamping, and Quasar dual-threshold consensus (BLS + Ringtail).
2. **KEM Interface**: Pluggable key encapsulation supporting ML-KEM-768, ML-KEM-1024, X25519, and hybrid modes.
3. **RNS Mesh Transport**: Medium-agnostic networking with hybrid PQ handshake (X25519 + ML-KEM-768 + ML-DSA-65) over LoRa, serial, TCP, and satellite.
4. **Sovereign KMS**: Enterprise key management with Shamir sharing, HPKE wrapping, transit engine, and TFHE bridge.
5. **Zero-Trust Fabric**: Identity-first overlay network with pluggable transport underlays and HSM integration.

1.2 What This Paper Adds

1. **QKD-KEM Bridge**: ETSI GS QKD 014 client wrapping QKD-derived keys into the existing `kem.KEM` interface.
2. **Triple-Hybrid Key Derivation**: $K = \text{HKDF}(\text{X25519_ss} \| \text{ML-KEM_ss} \| \text{QKD_key})$.
3. **Sensor Oracle**: New transaction type in QuantumVM for quantum sensor attestations with dual-threshold signatures.
4. **QRNG Integration**: Drop-in replacement of `crypto/rand` with quantum random number generator hardware.
5. **QKD Network Manager**: ITU-T Y.3800 compliant key pool orchestration across multiple QKD links.

2 Quantum Key Distribution Integration

2.1 QKD Protocol Support

The architecture supports all major QKD protocol families via the ETSI standardized hardware interface:

Protocol	Type	Range	Rate	Vendors
BB84	DV, prepare-measure	<100 km	∼1 Mbps	ID Quantique
CV-QKD	Continuous-variable	<50 km	∼10 Mbps	Toshiba
MDI-QKD	Measurement-device-indep.	<400 km	∼100 kbps	QuantumCTek
TF-QKD	Twin-field	<600 km	∼10 kbps	Toshiba, USTC
E91	Entanglement-based	<100 km	∼10 kbps	Research

Table 1: Supported QKD protocols via ETSI hardware interface.

2.2 ETSI GS QKD 014 Client

All QKD hardware interaction uses the ETSI GS QKD 014 REST API, the industry standard key delivery interface:

Listing 1: ETSI QKD 014 key request flow.

```
// Request quantum-derived key material
GET /api/v1/keys/{slave_SAE_ID}/status
  -> { "source_KME_ID": "kme-alpha",
       "key_count": 42,
       "max_key_size": 256 }

POST /api/v1/keys/{slave_SAE_ID}/enc_keys
  body: { "number": 1, "size": 256 }
  -> { "keys": [
         { "key_ID": "a1b2c3d4-...",
           "key": "base64(256-bit key)" }
       ]}

// Peer retrieves same key by ID
POST /api/v1/keys/{master_SAE_ID}/dec_keys
  body: { "key_IDs": ["a1b2c3d4-..."] }
  -> { "keys": [
         { "key_ID": "a1b2c3d4-...",
           "key": "base64(same 256-bit key)" }
       ]}
```

The ETSI 014 abstraction makes the architecture hardware-agnostic: BB84, CV-QKD, or TF-QKD appliances from any vendor present the same REST interface.

2.3 QKD-KEM Bridge

The QKD-KEM bridge implements the existing `kem.KEM` interface, enabling QKD to slot into any component that uses key encapsulation:

Listing 2: QKD-KEM bridge implementing existing KEM interface.

```
type QKDKEM struct {
    client *etsi014.Client
    peerID string
}

func (q *QKDKEM) Encapsulate() (ct, ss []byte, err error) {
    // Request key from local QKD node
    key, err := q.client.GetKey(q.peerID, 256)
    if err != nil { return nil, nil, err }
    // "ciphertext" is just the key_ID (peer fetches same key)
    return []byte(key.KeyID), key.Material, nil
```

```
}

func (q *QKDKEM) Decapsulate(ct []byte) (ss []byte, err error) {
    keyID := string(ct)
    key, err := q.client.GetKeyByID(q.peerID, keyID)
    return key.Material, err
}
```

This plugs directly into the RNS link handshake at `rns_link.go` where ML-KEM ciphertexts are exchanged. The QKD-KEM has zero ciphertext overhead—the key is delivered out-of-band by the quantum channel.

3 Triple-Hybrid Key Derivation

3.1 Security Model

Definition 3.1 (Triple-Hybrid Security)**.** A key exchange is triple-hybrid secure if the derived session key K is indistinguishable from random under the assumption that *at least one* of the following holds:

1. The Computational Diffie-Hellman problem on Curve25519 is hard (classical).
2. The Module-LWE problem with ML-KEM-768 parameters is hard (post-quantum algorithmic).
3. The laws of quantum mechanics prevent undetected eavesdropping on the QKD channel (physics-layer).

3.2 Protocol

Algorithm 1 Triple-Hybrid Key Exchange

Layer 1 — Classical ECDH:
  $\mathrm{ss}_1 \leftarrow \mathrm{X25519.DH}(\mathrm{ek}_A, \mathrm{epk}_B)$ (32 bytes)

Layer 2 — Post-Quantum KEM:
  $(\mathrm{ct}_2, \mathrm{ss}_2) \leftarrow \text{ML-KEM-768.Encaps}(\mathrm{epk}_B^M)$ (32 bytes)

Layer 3 — Physics-Layer QKD:
  $\mathrm{ss}_3 \leftarrow \mathrm{QKD.GetKey}(\mathrm{peer_ID}, 256)$ (32 bytes)
  $\mathrm{key_ID} \leftarrow \mathrm{QKD.KeyID}(\mathrm{ss}_3)$

Combine: $K \leftarrow \text{HKDF-SHA256}(\mathrm{salt}, \mathrm{ss}_1 \| \mathrm{ss}_2 \| \mathrm{ss}_3)$
Derive: $K_{\mathrm{send}}, K_{\mathrm{recv}} \leftarrow \text{HKDF-Expand}(K, \text{“triple-hybrid-v1”})$
Exchange: Send $\mathrm{epk}_A^X \| \mathrm{ct}_2 \| \mathrm{key_ID}$ to peer
Session: AES-256-GCM with derived keys

Theorem 3.2 (Triple-Hybrid IND-CCA2)**.** *If any one of X25519, ML-KEM-768, or the QKD key is secure (i.e., unknown to the adversary), then the HKDF output K is computationally indistinguishable from a uniformly random key of the same length. This follows from the extraction property of HKDF: given sufficient min-entropy in any one input, the output is pseudorandom regardless of the other inputs.*

3.3 Wire Format

Component	Classical	PQ Hybrid	Triple Hybrid
X25519 ephemeral PK	32 B	32 B	32 B
ML-KEM-768 ciphertext	—	1,088 B	1,088 B
QKD key_ID (UUID)	—	—	36 B
ML-DSA-65 signature	—	3,309 B	3,309 B
Total per-direction	**32 B**	**∼4.4 KB**	**∼4.5 KB**

Table 2: Wire overhead comparison: triple-hybrid adds only 36 bytes (UUID) over PQ hybrid.

The QKD layer adds only 36 bytes (a UUID key reference) to the handshake—the actual key material is delivered by the quantum channel, not the classical network.

4 Quantum Sensing and Attestation

4.1 Sensor Types

Sensor	Technology	Precision	Naval Application
Atomic clock	CSAC/optical lattice	<1 μs accuracy	GPS-denied timing
Magnetometer	SERF/NV-center	<1 fT sensitivity	Submarine detection
Gravimeter	Atom interferometry	<1 μGal	Subsurface mapping
Quantum radar	Microwave photons	Enhanced SNR	Target detection
Quantum lidar	Entangled photons	Sub-shot-noise	Terrain mapping

Table 3: Quantum sensor types with naval applications.

4.2 Sensor Oracle Architecture

Quantum sensor measurements are ingested into the Q-Chain via a Sensor Oracle transaction type:

Listing 3: Sensor attestation transaction.

```
type SensorAttestation struct {
    SensorType     SensorType   // AtomicClock, Magnetometer, etc.
    DeviceID       [32]byte     // Hardware attestation identity
    Timestamp      int64        // Sub-nanosecond precision
    Measurement    []byte       // Sensor-specific data format
    Uncertainty    float64      // Measurement uncertainty
    TEESignature   []byte       // TEE-signed device attestation
    MLDSASignature []byte       // ML-DSA-65 over all fields
}
```

The QuantumVM's existing `StampBlock` interface accepts arbitrary block IDs and messages—sensor attestations are stamped with the same Quasar dual-signature (BLS + Ringtail) that certifies consensus blocks.

4.3 Atomic Clocks for Consensus Timing

Chip-scale atomic clocks (CSAC) provide GPS-independent timing for DDIL environments:

- **Current**: QuantumVM uses `time.Now()` for block timestamps (OS clock, ∼1 ms accuracy).
- **With CSAC**: Sub-microsecond timestamp ordering via PTP/1PPS interface from quantum clock.
- **Byzantine detection**: Validators with drifted clocks produce timestamps outside the consensus window, enabling automatic identification of faulty or compromised nodes.
- **GPS-denied operation**: Aligns with RNS mesh networking for maritime/submarine scenarios where GPS signals are unavailable or jammed.

4.4 Magnetometers for Physical Security

Quantum magnetometers detecting sub-femtotesla fields enable:

- **Tamper detection**: Cryptographically attested magnetic field baseline for data center physical security. Any hardware modification changes the field signature.
- **Submarine detection**: Blockchain-attested magnetic anomaly data from distributed sensor arrays.
- **Navigation**: Magnetic field map matching for GPS-denied underwater navigation, attested on-chain for integrity.

4.5 Gravimeters for Subsurface Intelligence

Atom-interferometry gravimeters provide:

- **Subsurface mapping**: Detect underground structures, tunnels, and voids.
- **Immutable geological records**: Blockchain-attested gravity surveys with provenance chains.
- **Anti-spoofing**: Ground-truth gravity data anchored to consensus prevents adversary falsification.

5 QRNG Integration

5.1 Drop-In Entropy Source

Quantum random number generators (QRNG) provide entropy from quantum vacuum fluctuations or single-photon detection:

Listing 4: QRNG entropy source replacement.

```
// Current (QuantumSigner, line 104):
crypto.rand.Read(nonce)  // reads /dev/urandom

// With QRNG hardware:
qrng.Read(nonce)  // reads /dev/qrng0
// Fallback: crypto/rand if QRNG unavailable
```

Device	Interface	Rate	Standard
ID Quantique Quantis	USB/PCIe	4–240 Mbps	NIST SP 800-90B
Quside FQRNG	PCIe	400+ Mbps	NIST SP 800-22
Toshiba QRNG	Integrated	2 Gbps	MIST/BSI certified

Table 4: QRNG hardware options.

QRNG integration is the lowest-effort, highest-value component: a single import replacement at `rand.Read()` call sites across the key generation and nonce generation code paths.

6 Integration with Threshold MPC

6.1 QKD-Authenticated DKG Ceremony

Distributed Key Generation requires secure channels between all n parties. QKD provides information-theoretically secure channels for share distribution:

Algorithm 2 QKD-Authenticated Threshold Key Generation

Setup: Each pair (P_i, P_j) establishes QKD link
  $K_{ij} \leftarrow \text{QKD.GetKey}(P_j, 256)$ (symmetric key from quantum channel)

DKG Phase 1: Commitment broadcast
for each party P_i **do**
  Sample polynomial $f_i(x)$ of degree $t-1$
  Broadcast commitment $C_i = g^{f_i(0)}$ (public, signed ML-DSA-65)
  for each party P_j **do**
    Encrypt share: $e_{ij} = \text{AES-256-GCM}(K_{ij}, f_i(j))$
    Send e_{ij} via classical channel (QKD key protects confidentiality)
  end for
end for

Advantage: Share distribution is information-theoretically secure
Even a quantum computer cannot recover $f_i(j)$ from e_{ij} without K_{ij}

6.2 QKD-Secured Signing Sessions

Threshold signing messages between MPC parties use QKD-derived session keys:

- **CGGMP21**: Round messages encrypted with QKD keys via ZAP transport.
- **FROST**: Commitment and response shares protected by QKD-authenticated channels.
- **Ringtail**: Lattice threshold shares doubly protected—PQ algorithmic + QKD physics-layer.
- **BLS**: Aggregate signature shares transmitted over QKD-secured links.

6.3 Key Rotation with QKD Refresh

Ringtail epoch rotation (every 10 minutes) can incorporate QKD key refresh:

1. At epoch boundary, validators request fresh QKD keys from all peer links.

2. New Ringtail threshold keys generated using QKD-authenticated DKG.
3. Old keys destroyed; forward secrecy guaranteed by both ephemeral PQ keys and QKD one-time keys.
4. Triple protection: even if lattice assumptions fail AND classical crypto breaks, QKD keys remain secure.

7 Integration with Consensus

7.1 QKD-Authenticated Validator Links

Quasar consensus samples k peers per round. QKD provides an additional authentication layer:

- **Pre-established QKD links**: Validators with fiber-connected QKD appliances establish shared keys.
- **Consensus message authentication**: HMAC using QKD-derived key appended alongside BLS signatures.
- **Sybil resistance**: QKD links are physical—an adversary cannot create virtual QKD connections without quantum hardware and fiber access.
- **Partial deployment**: Validators without QKD hardware continue using PQ-hybrid authentication. The system degrades gracefully.

7.2 Sensor-Attested Block Production

Blocks can include quantum sensor attestations as special transactions:

- Atomic clock timestamps replace OS timestamps for sub-microsecond ordering.
- Magnetometer readings provide physical-layer environmental context.
- All sensor data receives Quasar dual-signature (BLS + Ringtail) for immutable provenance.
- Sensor attestation chain provides tamper-evident physical security monitoring.

8 KMS Integration

8.1 QKD Key Import

The Sovereign KMS Transit Engine supports key import via `importKeyMaterial()`:

- QKD-derived keys imported as a new key version type: `hanzo-qkd:v{N}:{base64}`.
- Keys stored with the same per-org isolation as algorithmic keys.
- QKD keys can serve as root keys for Shamir splitting, providing physics-layer root-of-trust.
- HPKE wrapping extended to include QKD key material alongside ML-KEM-768.

8.2 Key Hierarchy with QKD Root

Layer	Key Source	Security Basis	Rotation
Root	QKD + Shamir	Physics + threshold	Ceremony
Organization	AES-256 (QKD-derived)	Physics-layer	Policy
Session	Triple-hybrid KDF	Classical + PQ + physics	Per-session
Message	ML-KEM ephemeral	Algorithmic PQ	Per-message

Table 5: Key hierarchy with QKD-anchored root of trust.

9 Zero-Trust Fabric Integration

9.1 QKD Transport Underlay

The Hanzo ZT fabric supports pluggable transport underlays. A QKD-authenticated underlay implements the existing `Transport.Connection` interface:

- **Physical link**: Fiber-optic QKD channel between zero-trust nodes.
- **Key delivery**: ETSI GS QKD 014 provides symmetric keys.
- **Data encryption**: AES-256-GCM using QKD-derived keys on the classical channel.
- **Authentication**: QKD keys authenticate zero-trust identity assertions.
- **Policy enforcement**: Zero-trust policies applied after QKD authentication—QKD provides the channel, ZT provides the authorization.

10 Standards Compliance

Standard	Scope	Compliance
ETSI GS QKD 004	Session-based key delivery	Client implementation
ETSI GS QKD 014	REST key delivery API	Client implementation
ITU-T Y.3800	QKD network overview	Functional architecture
ITU-T Y.3802	QKD functional architecture	Key management layer
NIST SP 800-90B	Entropy source validation	QRNG qualification
FIPS 203 (ML-KEM)	Key encapsulation	Production (existing)
FIPS 204 (ML-DSA)	Digital signatures	Production (existing)
FIPS 205 (SLH-DSA)	Hash-based signatures	Production (existing)
CNSA 2.0	NSA algorithm suite	Full compliance

Table 6: Standards compliance matrix.

11 Naval Deployment Scenarios

11.1 Shore-to-Ship QKD via Submarine Fiber

- QKD appliances at shore facility and pier-side ship terminal.
- Pre-load QKD key pool before departure (keys consumed during deployment).
- Triple-hybrid key derivation for all ship-to-shore encrypted communications.
- Key pool size determines operational endurance without QKD refresh.

11.2 Satellite QKD for Global Coverage

- Satellite-based QKD (demonstrated by Micius satellite at 1,200 km) for global key distribution.
- Trusted-node relay architecture for intercontinental links.
- QKD keys distributed to fleet units via satellite pass windows.
- Stored keys consumed for consensus authentication between passes.

11.3 Submarine Quantum Sensing Array

- Distributed quantum magnetometer array for ASW (anti-submarine warfare).
- Sensor attestations transmitted via RNS mesh (acoustic/ELF radio).
- Blockchain-anchored magnetic anomaly data for post-mission analysis.
- Tamper-evident sensor chain prevents adversary data injection.

11.4 GPS-Denied Navigation

- Quantum atomic clocks provide autonomous timing without GPS.
- Quantum gravimeters enable terrain-referenced navigation underwater.
- Quantum magnetometers support magnetic-field navigation.
- All sensor data attested on Q-Chain with dual-threshold signatures.

12 Implementation Roadmap

Phase	Component	LOC	Timeline
1	QRNG entropy integration	∼200	Q2 2026 (planned)
2	ETSI GS QKD 014 client	∼800	Q3 2026 (planned)
3	QKD-KEM bridge	∼400	Q3 2026 (planned)
4	Triple-hybrid key derivation	∼300	Q3 2026 (planned)
5	Sensor Oracle (QuantumVM)	∼1,000	Q4 2026 (planned)
6	QKD Network Manager	∼1,500	Q1 2027 (planned)
7	Zero-trust QKD underlay	∼800	Q1 2027 (planned)
	Total	**∼5,000**	

Table 7: Implementation roadmap: 5,000 lines of Go leveraging existing interfaces.

All components are additive—they implement existing interfaces (`kem.KEM`, `Transaction`, `crypto/rand`, `Transport.Connection`) without modifying production code.

13 Competitive Analysis

Capability	**Ours**	**QAN**	**IOTA**	**Algorand**
PQ signatures (prod)	✓	Planned	–	Partial
PQ key exchange (prod)	✓	–	–	–
Threshold PQ (Ringtail)	✓	–	–	–
QKD integration	Design	–	–	–
Quantum sensing	Design	–	–	–
Triple-hybrid KDF	Design	–	–	–
Mesh/DDIL transport	✓	–	Partial	–
Zero-trust fabric	✓	–	–	–

Table 8: Competitive landscape: no competitor has PQ consensus, let alone QKD/sensing.

14 Conclusion

We have presented an architecture for integrating quantum key distribution and quantum sensing into a production post-quantum blockchain stack. The triple-hybrid key derivation—combining classical ECDH, lattice-based KEM, and physics-layer QKD—provides defense-in-depth across three independent security foundations.

The architecture is uniquely feasible because the production stack already provides the integration surfaces: a pluggable KEM interface for QKD keys, a QuantumVM for sensor attestations, a mesh transport for DDIL deployment, and a zero-trust fabric for QKD-authenticated channels. The estimated 5,000 lines of new code leverage these existing interfaces without modifying production systems.

No competing blockchain or military networking system offers this combination of production post-quantum cryptography, QKD integration architecture, and quantum sensor attestation—let alone over a delay-tolerant mesh network operating on LoRa, packet radio, and satellite links.

References

[1] NIST, "ML-KEM Standard," FIPS 203, 2024.

[2] NIST, "ML-DSA Standard," FIPS 204, 2024.

[3] NIST, "SLH-DSA Standard," FIPS 205, 2024.

[4] NSA, "CNSA Suite 2.0," 2022.

[5] ETSI, "QKD; Protocol and data format of REST-based key delivery API," GS QKD 014 V1.1.1, 2019.

[6] ETSI, "QKD; Application Interface," GS QKD 004 V2.1.1, 2020.

[7] ITU-T, "Overview on Networks Supporting Quantum Key Distribution," Y.3800, 2019.

[8] ITU-T, "Quantum Key Distribution Networks – Functional Architecture," Y.3802, 2020.

[9] NIST, "Recommendation for the Entropy Sources Used for Random Bit Generation," SP 800-90B, 2018.

[10] C. Bennett, G. Brassard, "Quantum Cryptography: Public Key Distribution and Coin Tossing," TCS 2014 (orig. 1984).

[11] M. Lucamarini *et al.*, "Overcoming the Rate-Distance Limit of QKD without Quantum Repeaters," Nature 557, 2018.

[12] S.-K. Liao *et al.*, "Satellite-to-Ground Quantum Key Distribution," Nature 549, 2017.

[13] "Quantum Secured Blockchain Framework," Nature Scientific Reports, 2025.

[14] "Protecting Distributed Blockchain with Twin-Field QKD," arXiv:2603.14826, 2026.

[15] "Practical Two-Round Threshold Signature from LWE," IACR ePrint 2024/1113.

[16] Hanzo Team, "Quasar Consensus," Lux Industries, 2025.

[17] Infleqtion, "Tiqker Quantum Atomic Clock," CES 2026.

[18] A. Shamir, "How to Share a Secret," CACM, 1979.